1. Thay đổi phần cứng RTL

Baud được ghi lúc chạy; IRQ được tạo từ trạng thái UART và đưa ra component.

File / trạng thái	Vị trí thay đổi trong source	Nội dung và mục đích
rtl/baud_rate_gen.v	L6: input wire [15:0] divider_i	Thêm divider_i 16 bit và reload_i; bộ đếm 16 bit xuất tick_tx và tick_rx cùng nhịp ×16 theo divider hiện tại. Khi APPLY: reset bộ đếm/tick; divider 0 dừng tick.
rtl/register_file.v	L25: REG_FIFO=12'h014	Thêm 0x18 SHADOW, 0x1C ACTIVE, 0x20 CONTROL_STATUS. Thêm baud_safe và APPLY; chỉ đổi ACTIVE khi TX_DONE, RX an toàn và divider khác 0. Ghi byte lane 0/1 vào SHADOW.
rtl/register_file.v	L42: wire [2:0] int_status	Thêm INT_ENABLE 0x24, INT_STATUS 0x28 và irq_o. Bit 0 RX có dữ liệu; bit 1 TX_DONE; bit 2 RX overrun. IRQ = OR(enable & status), reset tắt toàn bộ enable.
rtl/dti_uart.v	L29: reg [4:0] rx_quiet_ticks	Thêm bộ đếm xác nhận RX idle 16 tick; nối baud_div và baud_apply giữa register_file và generator. Đưa IRQ ra top UART; parameter BAUDRATE giữ vai trò mặc định sau reset.
rtl/uart_rx/rx_topmodule.v	L32: assign baud_idle_o	Xuất baud_idle_o khi RX controller idle, RX đã đồng bộ ở mức cao và không có start edge đang phát hiện. Đây là đầu vào cho điều kiện đổi baud an toàn.
rtl/uart_avalon.v	L16: output wire tx, rts_n, irq	Bổ sung cổng irq, nối từ dti_uart ra giao diện component. Đường đọc/ghi Avalon → bridge → APB vẫn giữ nguyên.

ĐIỂM CẦN NHỚ

Luồng baud: C → SHADOW → APPLY hợp lệ → ACTIVE → baud_tick → TX/RX.

Luồng IRQ: trạng thái RX/TX/overrun → mask → irq → Nios V. Peer phải ngừng gửi khi đổi baud.

2. Thay đổi firmware C

ISR phục vụ RX nhanh; main giữ phần xử lý dòng và điều phối baud/TX.

File / trạng thái	Vị trí thay đổi trong source	Nội dung và mục đích
firmware/dti_uart.h	L8: DTI_BAUD_SHADOW	Bổ sung offset 0x18–0x28, các mask baud/ngắt và khai báo API dti_baud_divider(), dti_set_baud().
firmware/dti_uart.c	L90: int dti_baud_divider	Tính divider gần nhất bằng số học 64 bit; kiểm tra 1..65535 và sai số ≤2%. dti_set_baud() ghi SHADOW, kiểm tra BUSY, APPLY và xác nhận ACTIVE, có timeout.
firmware/uart_irq.c [MỚI]	L10: int uart_irq_init	File mới: đăng ký ISR, bật RX/overrun; ISR lấy tối đa 16 byte kèm parity vào ring 512 entry (dùng được 511). TX_DONE được ghi nhận rồi mask; tràn lưu lỗi và tắt IRQ UART.
firmware/uart_irq.h [MỚI]	L6: int uart_irq_init	File mới: giao diện ISR, nhận từ ring, đọc lỗi, arm/nhận thông báo TX hoàn tất. ISR độc quyền pop RX phần cứng; main độc quyền enqueue TX.
firmware/uart_platform.h	L19: static inline uint32_t uart_irq_lock	Thêm wrapper khóa/khôi phục IRQ và đăng ký qua HAL alt_ic_isr_register(). Nếu BSP thiếu IRQ UART thì trả lỗi, không tự chuyển về polling.
firmware/main.c	L30: static int baud_command	Thêm lệnh JTAG :baud N. Init đăng ký IRQ; poll đọc ring ISR và cờ TX hoàn tất. Sau khi queue TX rỗng, kiểm tra RX rồi APPLY trong critical section ngắn; tăng timeout theo baud thấp.

ĐIỂM CẦN NHỚ

Vị trí thêm trong main.c: receive_char() nhận lệnh JTAG; uart_demo_init() gọi uart_irq_init(); uart_demo_poll() dùng ring/cờ ISR và xử lý pending_baud. Chat Enter/CRLF/Backspace giữ hành vi cũ; JTAG console tiếp tục polling.

3. Kết nối hệ thống và công cụ chạy

Cần sinh lại hệ thống/BSP và dùng SOF + ELF mới tương thích.

File / trạng thái	Vị trí thay đổi trong source	Nội dung và mục đích
fpga/ip/uart_avalon_hw.tcl	L55: add_interface irq interrupt sender	Khai báo interrupt sender, gắn clock/reset và associatedAddressablePoint avalon. Không thêm chân GPIO IRQ ra board.
fpga/scripts/create_system.tcl	L56: add_connection cpu.platform_irq_rx uart.irq	Nối UART IRQ vào cpu.platform_irq_rx và gán irqNumber 1. JTAG UART vẫn ở IRQ 0.
firmware/build.sh	L11: sources="$firmware_dir/main.c	Thêm uart_irq.c vào ứng dụng echo/chat. Scope và selftest giữ nguồn polling, không cần dùng ISR chat.
firmware/terminal_uart.sh	L8: baud="${2:-115200}"	Thêm đối số baud thứ hai, mặc định 115200; truyền baud cho picocom. Ví dụ: terminal_uart.sh /dev/ttyUSB0 9600.
firmware/serial_echo_test.py	L11: parser.add_argument('--baud'	Thêm --baud để kiểm tra serial ở tốc độ mới; điều chỉnh thời hạn đọc/ghi theo baud.
File sinh tự động: uart_soc.qsys, uart_soc.sopcinfo, firmware/build/bsp/system.h	Sinh từ create_system.tcl và firmware/build.sh	Generate lại Platform Designer và BSP; system.h có UART_IRQ=1, controller ID=0. Các file này không sửa tay. bsp_settings.tcl chỉ cập nhật chú thích về IRQ/mtime.

Các khối giữ nguyên thuật toán: Avalon–APB bridge, APB slave, FIFO và TX/RX controller/datapath. RX top chỉ bổ sung idle; pin assignment không đổi.

ĐIỂM CẦN NHỚ

Thao tác: dừng gửi picocom → gõ :baud 9600 tại JTAG → chờ báo thành công → mở lại picocom ở 9600.

Mặc định/reset: 115200. Driver từ chối 230400 vì sai số khoảng −3,12%; không có auto-baud.

4. File kiểm thử và phạm vi xác nhận

Bảng này tóm tắt các nhóm kiểm thử đã bổ sung hoặc điều chỉnh.

File / trạng thái	Vị trí thay đổi trong source	Nội dung và mục đích
tb/tb_baud_dynamic.sv [MỚI]	L2: module tb_baud_dynamic	File mới: divider 1/2/65535, reload, divider 0 và đổi divider; 263.816 checks.
tb/tb_baud_irq.sv [MỚI]	L3: module tb_baud_irq	File mới: serial nhiều baud, APPLY khi CTS/RX bận hoặc có start edge, byte strobe, IRQ mask/level, parity/overrun và reset; 269 checks.
firmware/tests/test_driver.c; test_demo.c; test_irq.c [test_irq.c MỚI]	Các bài test C native; chạy bởi firmware/test_driver.sh	Kiểm tra tính baud, timeout, lệnh JTAG; ISR thật trong model host, ring wrap/overflow, parity, mask TX_DONE và lỗi đăng ký ISR.
tb/firmware_cosim.c; tb_firmware_cosim.sv; tb_firmware_echo_cosim.sv	Co-simulation C + RTL; các script run_firmware_*cosim.sh	Driver + RTL: 768 frame, đổi baud bằng API. Chat + RTL: 538 byte, IRQ RTL được model host gọi ISR C; đổi 9600 → 115200, từ chối 230400.
tb/tb_niosv_firmware.sv; sim/run_tests.sh	Giám sát CPU thật; danh sách regression	Cho phép trap ngắt hợp lệ, kiểm tra có IRQ entry; payload chat dùng ASCII. Thêm hai test RTL mới; cập nhật các TB cũ cho cổng mới và địa chỉ không hợp lệ.
docs/BAUD_INTERRUPT.md; firmware/VALIDATION.md; README và tài liệu tích hợp	Hướng dẫn sử dụng / kết quả đã ghi nhận	Ghi register map, quy trình đổi baud, chính sách ISR và kết quả. Tài liệu tóm tắt này không chạy lại các kiểm thử.

ĐIỂM CẦN NHỚ

Đã ghi nhận: RTL 15/15 PASS; C/ISR và C+RTL PASS; ELF build PASS; Quartus timing +0,148 ns.
Questa compile-only PASS; chạy CPU bị chặn socket. SOF còn Simulation-Only trong môi trường công cụ; chưa xác nhận bản mới trên kit.